FH Salzburg
MultiMediaTechnology

Basics Game Development

Introduction

Technology
Health
Media

Team

- Radomir Dinic
  Lectures and Labs

- Samuel Arzt
  Labs

Content (Curriculum)

- **Game loop** (initialisation and loading of resources, time-based update, controlled shutdown);
- **2D graphics skills** (sprites, sprite animation, tile maps, occlusions);
- **input methods** (e.g. keyboard, mouse);
- **basic affine transformations and coordinate spaces** (e.g. for manipulating objects or game camera);
- **2D collision detection** (e.g. AABB, circle);
- **scene management**;
- **graphical user interface** (e.g. menus, heads-up displays);
- exercises based on an existing framework (C# / SFML).

Learning Outcomes:

The students…

- can explain the basic structures and processes of a game engine in their own words and structure and develop simple interactive applications
- can programmatically implement simple, interactive 2D games based on an existing game engine and cleanly design the software architecture and program flow in the code
- use 2D graphics for representation (e.g. animated sprites, tile maps, GUI) and can correctly implement occlusions.
- can switch between coordinate spaces and use vectors and affine transformations to manipulate objects and camera viewpoints.
- can implement simple methods of 2D collision detection to realize game logic

Workload

- **2,5 ECTS**
  - 62,5 hours workload in total
- **2 SWS**
  - 7 * 2 * 45 min Lectures = 10,5 hours
  - 7 * 2 * 45 min Lab-Sessions = 10,5 hours
- **41,5 hours for assignments and autonomous studying (~4-5h weekly)**

This should be the average workload for an average student to get an average grade.

Grading

- **60% Lecture:**
  - **Final Exam:** 90-minute written exam; **Date:** 25.04.2025
- **40% Lab:**
  - **Quizzes:** 10-minute quizzes in every lab session

Quiz Details

- **Format:** Multiple-choice (conducted in MS Teams)
- **Content:**
  - General coding-related questions
  - After assignment submission deadlines, quizzes focus on the assignment topics in the following lab session

Assignments

3 „bigger“ assignments in MS-Teams – are not graded but evaluated through quizzes.

- Basic SFML Setup (Game-Loop, Transformations, Time)
- Basic Engine (Asset handling, Sprite Animation, Sound, Input)
- Final Engine (Components, Collisions and Basic Physics)
  graded assignment

Requirement for MMP1 Game!

Sources

- **Core Literature**
  - Game Coding Complete 4th edition by McShaffry and Graham

- **Additional Literature**
  - https://github.com/miloyip/game-programmer
  - https://gameprogrammingpatterns.com/
  - Game Engine Architecture 3rd edition by Jason Gregory

- **Videos**
  - http://www.cs.mun.ca/~dchurchill/teaching.shtml
  - https://www.youtube.com/channel/UCQ-W1KE9EYfdxhL6S4twUNw
  - https://www.youtube.com/channel/UC0JB7TSe49lg56u6qH8y_MQ

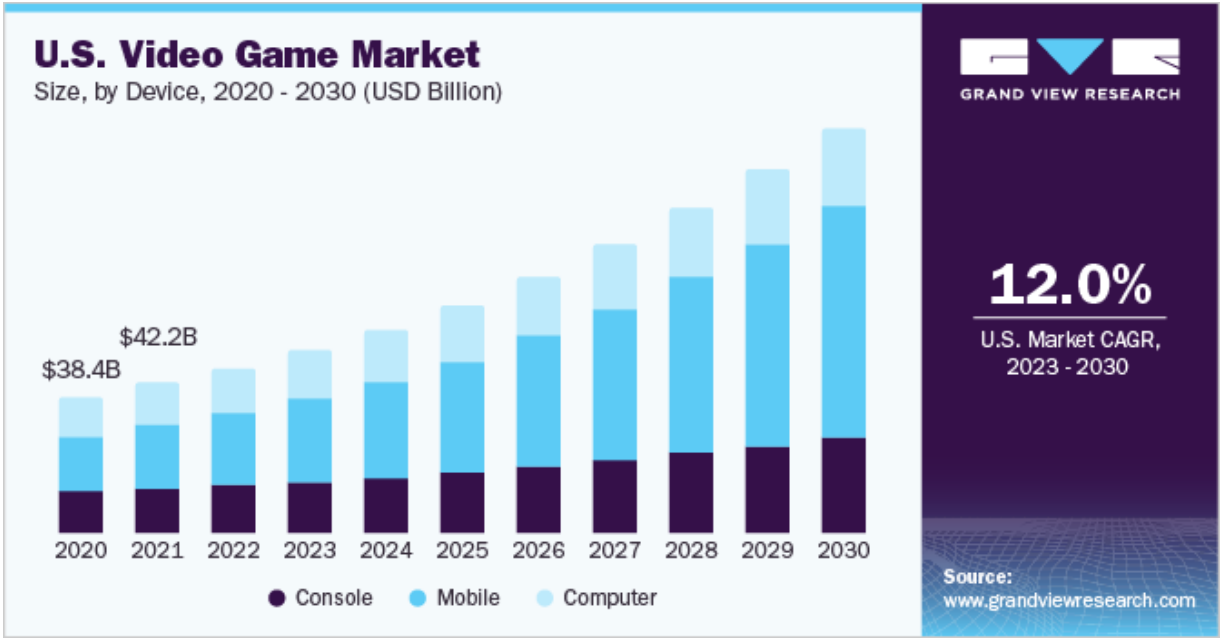
Why to become a game developer?

Why to become a game developer?

- Use your creative and analytical skills.
- Work in an enthusiastic and casual environment.
- Enjoy a steady career in a dynamic industry.
- Earn a competitive salary.
- Make a career out of helping people have fun.
- International job opportunities.
- The possibility to easily change the branch of industry if you want (very welcome everywhere).

📕 Additional Reading:
Game Coding Complete, Page 1-23,
What Is Game Programming Really Like?

Market Growth Over Time



Average Salary

- Talent.com (USA)

- Devjobs.at (Austria)

What thinks AI about becoming a game developer? (o3-mini with Web-Search)

- **Explosive Market Growth:** The gaming industry is booming—with forecasts predicting it could exceed a $300 billion valuation by 2025/2026—offering tremendous career and financial opportunities.
- **Diverse Career Paths:** Game development isn't just about coding; it spans art, design, storytelling, audio, and production, letting you pursue your unique passion.
- **Skill Enhancement:** Developing games hones critical problem-solving, programming, and design skills that are valuable across multiple tech sectors.
- **Creative Expression:** You get to bring your creative visions to life, crafting immersive worlds and narratives that resonate with millions of players.
- **Innovative Technologies:** With emerging tools like AI and advanced engines, developers can push creative boundaries and streamline production processes.
- **Global Collaboration:** The industry's vibrant community offers networking, mentorship, and collaborative projects—fueling both personal growth and industry advancement.
- **Resilience & Future-Proofing:** Despite periodic industry challenges, the sustained demand for innovative gaming experiences ensures long-term career stability and growth.

Game ♥ Web

GameReel 2021

Let's ask AI again: Web || Game (o3-mini + Web)

Both majors have strong career prospects—but your choice should align with your interests and career goals.

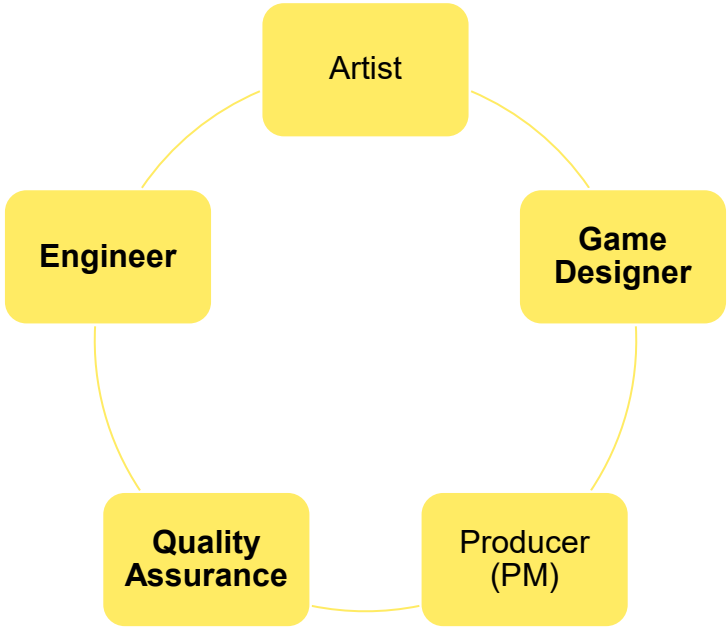
- **Web Major:**
  • Focuses on backend and frontend development, mobile technologies, and interactive web design.
  • Opens doors to roles in software development, digital marketing, and web/mobile app creation.
  • Ideal if you enjoy creating versatile, user-friendly digital solutions and working with evolving internet technologies.
- **Game Major:**
  • Concentrates on game development, computer graphics, and mixed reality.
  • Prepares you for roles in the gaming industry, simulation, and immersive media.
  • Best if you’re passionate about interactive experiences, storytelling through games, and cutting-edge simulation technology.

Ask yourself which subjects excite you more day-to-day. If you see yourself building dynamic web applications or working in the fast-evolving tech sector, the Web track might be the way to go. If you’re drawn to creative, interactive experiences and the gaming world, the Game track could be a perfect match.

Both options benefit from practical, project-based learning and strong industry connections at FH Salzburg. **Consider speaking with current students** or alumni to get additional insights.

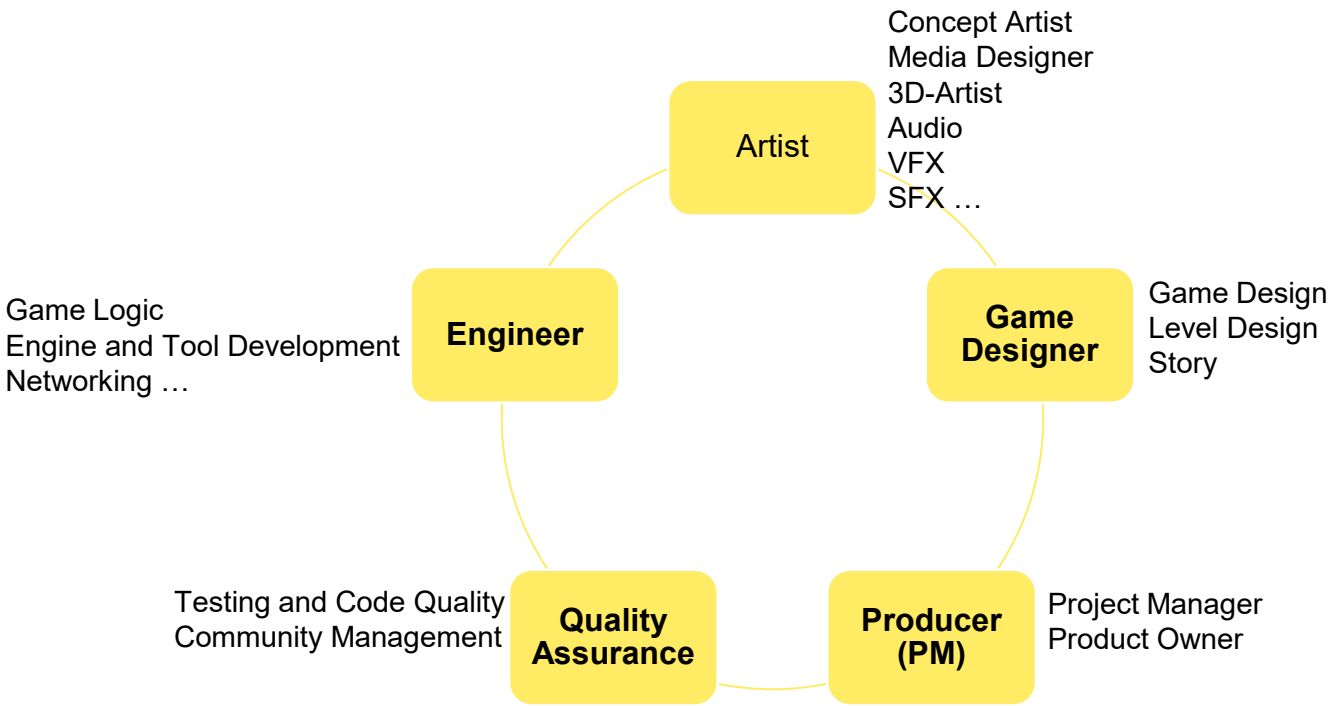
The World of Game Development

Who works on a Game-Project?



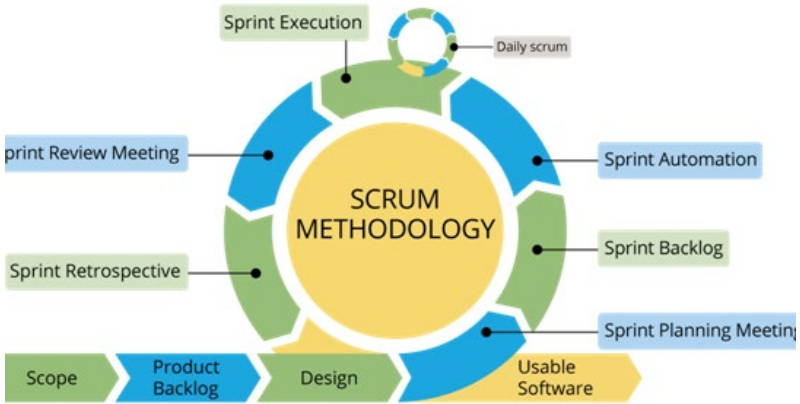
Examples?

Who works on a Game-Project?

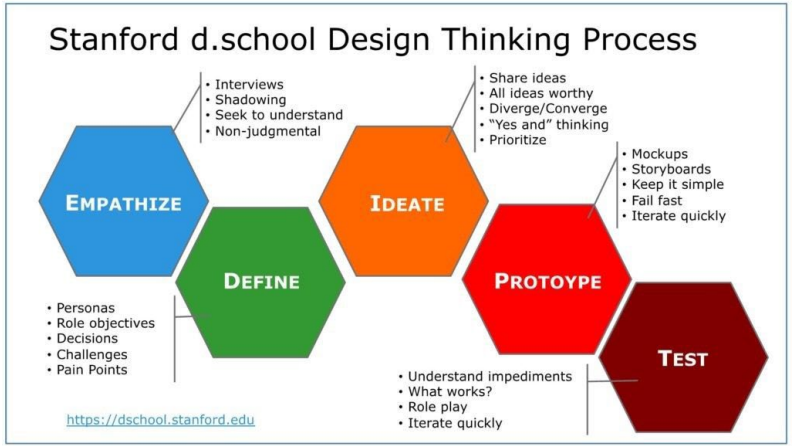


Common (Agile) Workflow Models

SCRUM



Design Thinking



7 Stages of Game Development

Taken from https://www.g2.com/articles/stages-of-game-development

What‘s in a Game?

Gameplay	Engine
Game World	User Interface
Game Mechanics	Animation
Characters	Graphics
Quest	Input
Atmosphere	Network
Story	Artificial Intelligence
	Physics
	Audio

Game Engines

The Wikipedia Definition

A **game engine** is a software framework primarily designed for the development of video games, and generally includes relevant libraries and support programs.[1] The "engine" terminology is similar to the term "software engine" used in the software industry.

Game engine can also refer to the development software utilizing this framework, typically offering a suite of tools and features for developing games.[2][3]

Developers can use game engines to construct games for video game consoles and other types of computers. The core functionality typically provided by a game engine may include a rendering engine ("renderer") for 2D or 3D graphics, a physics engine or collision detection (and collision response), sound, scripting, animation, artificial intelligence, networking, streaming, memory management, threading, localization support, scene graph, and video support for cinematics. Game engine implementers often economize on the process of game development by reusing/adapting, in large part, the same game engine to produce different games[4] or to aid in porting games to multiple platforms.

Game Engines Summary

A Game Engine is a software product that contains a set of libraries, tools and programs helping developers and designers working together on creating interactive realtime content for different platforms.

- Core components of a game engine:
  - Graphical Level Editor
  - Physics Engine
  - Render Engine
  - Audio Engine
  - Animation Engine
  - Input Management
  - Asset Management
  - …

Game Engines – Applications

- Video Games
- Mixed Reality
- Mobile Applications
- Web Applications
- Digital Video Production

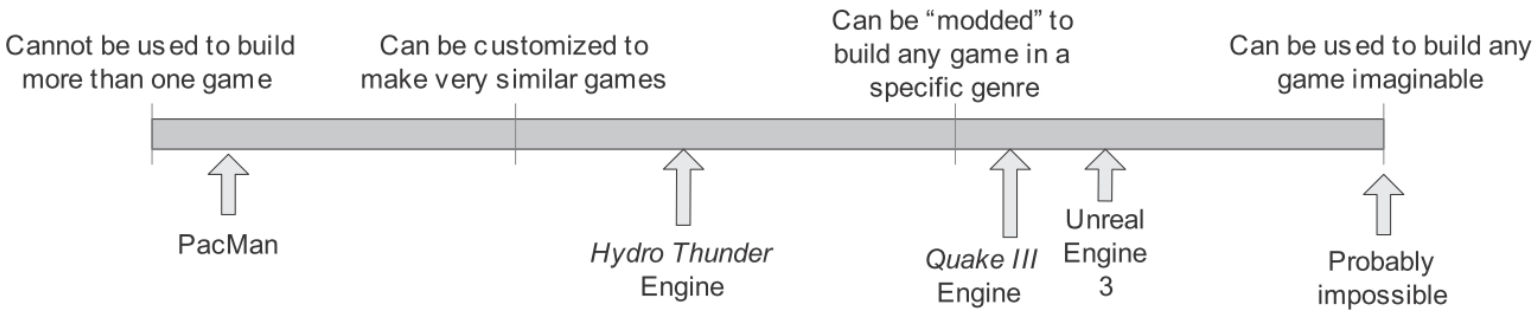
Popular „General Purpose Game Engines“ - Comparison

	Installation & Ownership	2D/3D	Ease of Use	Integration & Compatibility	VR Support	Customer Support
Unreal Engine	***	Both	***	******	Yes	***
Amazon Lumberyard	***	3D Only	*****	***	Yes	***
CryENGINE	***	Both	***	******	Yes	***
Unity	***	Both	***	*****	Yes	***
GameMaker: Studio	******	2D Only	*****	***	No	***
Godot	*****	BothBoth	*****	***	No	*****
Cocos2d	*****	2D Only	*****	***	No	***

Wide Distribution

Source: https://www.incredibuild.com/blog/top-7-gaming-engines-you-should-consider-for-2020

Reusability of Engines



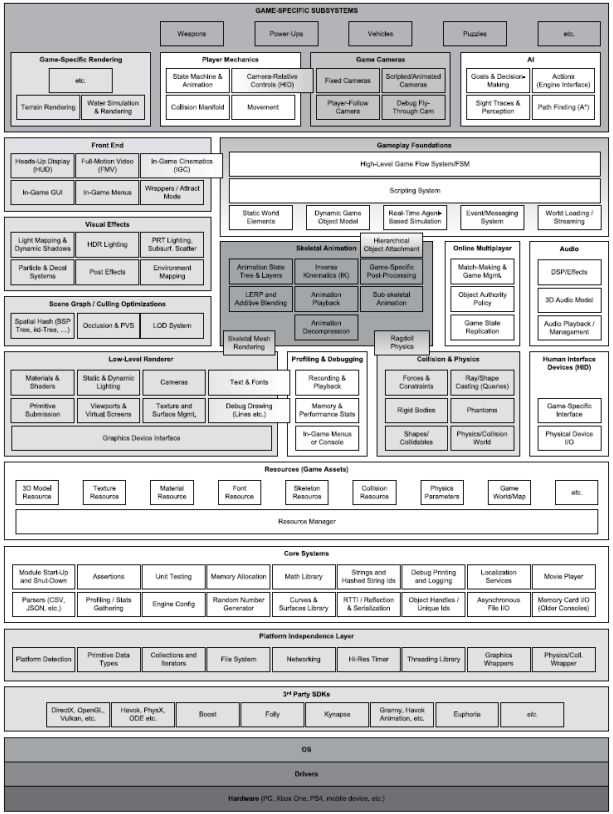
📕 Additional Reading:
Game Engine Architecture, Chapter 1.3, What is a Game Engine?

Game Architecture

Games / Game Engines are big and complex systems.

- How can we reduce complexity?
  - Divide and conquer.
  - Many subsystem and layers form a complex system.
  - Easier to test, debug and reuse.
- Challenges
  - How to split the game in subsystems.
  - How should subsystems communicate and interact with each other.

Game Engine Architecture



e 1.16. Runtime game engine architecture.

📕 Additional Reading:
For a detailed overview see Game Engine Architectures Figure 1.16. “Runtime game engine architecture”.

Game Engine Layer Architecture

Game Play (Game Objects, Level, Logic...)
Application (Initialization, Loop, Shutdown)
Engine Subsystems (Physics, Audio, Renderer...)
Support Layer (Input, Math, ...)
3rd Party SDK (OpenGL, SFML...)
System (OS, Driver, Hardware...)

System

OS, Driver, Hardware(PC, Console, Mobile, Web)

- Windows
  - Game must share hardware with multiple running programs.
- Consoles
  - Thin OS layer.
  - Game has nearly exclusive access to hardware.
  - Latest consoles have more limitations for games. (eg. Significant processing power and RAM is reserved for OS.
- Mobile / Web
  - Stricter limitations from OS/Browser.
  - OS forces the game to suspend while in background.

3rd Party SDK

OpenGL/DirectX, SFML (Media), Box2D (Physics), FMod (Audio), …

There are many engines/middleware/libraries useful for games (C#)

- Graphics Library
  - SDL, SFML, MonoGame…
- Physics Library
  - Box2D, Bullet, …
- Audio
  - Naudio, FMod, …
- Network
  - .NET,

Support Layer

Debug Tools, Input, Math, Thread, FileSystem, Memory, Network

- Abstracts Operation System
  - Input
  - FileSystem
  - Thread
  - Network
- Useful utility functions
  - Math Utils (eg. Vector classes)
  - Debug tools (Logger, Asserts, Profiler,…)
  - Others (String Utils, Data collections)

Engine Subsystems

AI, InputManager, SceneManager, GUI, Physics, Renderer, …

- The heart of the engine separated in loosely coupled systems.
- Not every game needs all sub systems

Game Runtime

GamePlay: GameObjects, Level, HUD, Widgets…

GameStateManager: MenuState, LoadingState, MainState,…

Application: Initialization, Game Loop, Game Shutdown

- Application: Starts, runs and stops the game
- Game State Manager: Every game can be expressed as finite state machine (Wikipedia).
- Game Play: Game Objects define the actors and objects in the game.

Game Application Layer

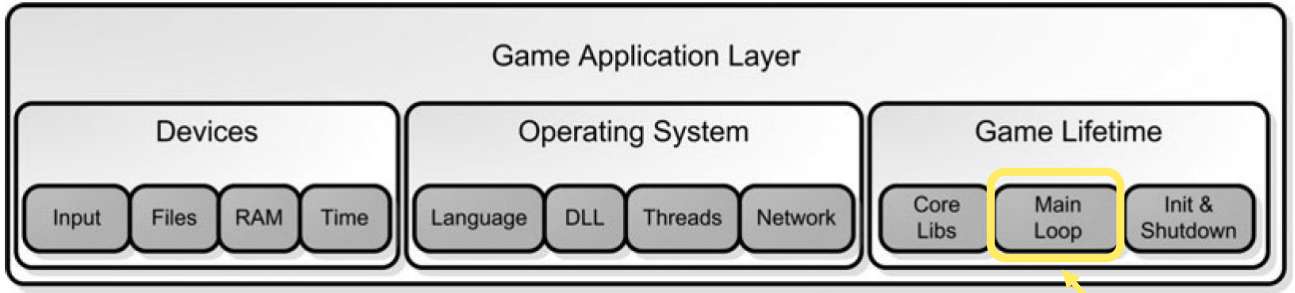


Figure 2.2
A closer look at the application layer. Taken from Game Code Complete

Details in „The Game Loop“ Section

- Reading Input
- File System and Resource Caching
- Managing Memory
- Init, Main Loop, Shutdown

Game Logic

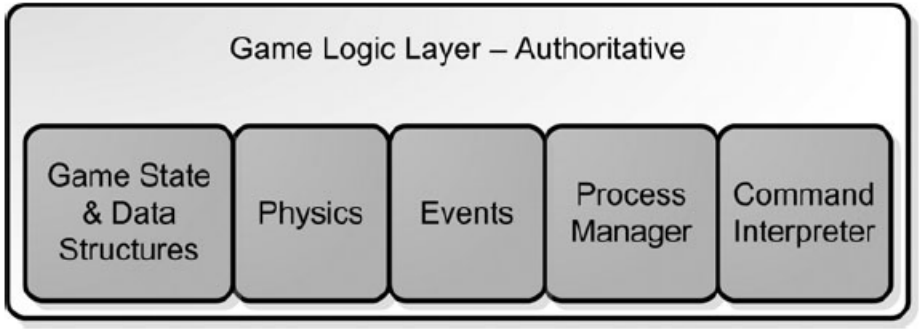


Figure 2.3
Game logic and its subsystems. Taken from Game Code Complete

Game View – Human Perspective

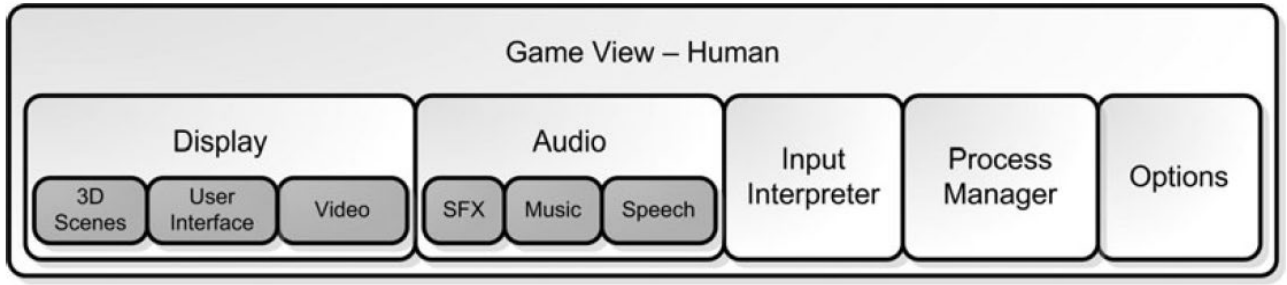


Figure 2.4
Subsystems that create a game view for a human player. Taken from Game Code Complete

- Your Artists Need to Know What Your Engine Can Do
- “Usability in Games” is a tough topic (HCI topic)
- Dedicated Process Manager for Interface-Actions (Animation, Streaming, …)

SFML – Simple and Fast Multimedia Library

- SFML is not a game engine
- SFML (Simple and Fast Multimedia Library) is a Library that provides a simple interface to the various components of your PC. It is composed of five modules:
  - System, Window, Graphics, Audio, Network
- It is multi platform:
  - Windows, Linux, macOS (soon Android and iOS)
- It is multi language:
  - Official for C++ and C#
  - Many other unofficial bindings (Java, Ruby, Python, Go, Rust, …)

Practical Part of this Course

- In order to increase the understanding of the internal processes of a game engine, we will build a simplified game engine in this course.
- Editor: VS-Code
- Programming Language: C#
- Media Library: SFML

This small engine is also the basis for your MMP1 if you plan to choose the “Mixed Reality and Game” major.

Engine for MMP1

What it can do:

- Easy to extend (if using component-based architecture)
- Sprites
- Basic 2D physics (collisions, triggers for gameplay)
- GUI (inventory, health, mana, score, start screen, …)
- Basic Input for Mouse/Keyboard
- Audio

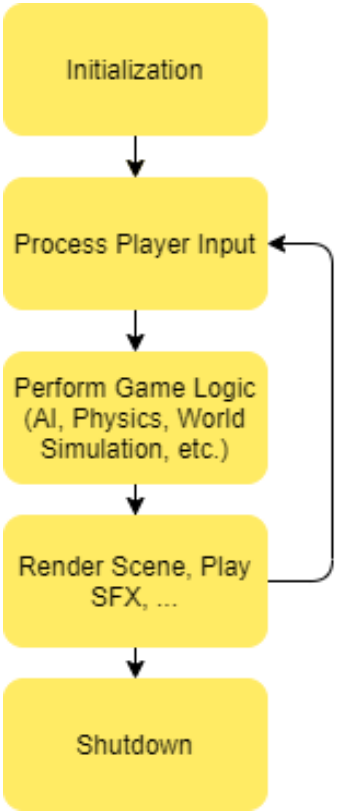
The Game Loop

Game Loop

The **game loop** is the overall flow control for the entire game program. It's a loop because the game keeps doing a series of actions over and over again until the user quits. Each iteration of the game loop is known as a **frame**. Most real-time games update several times per second: 30 and 60 are the two most common intervals. If a game runs at 60 FPS (**frames per second**), this means that the game loop completes 60 iterations every second.

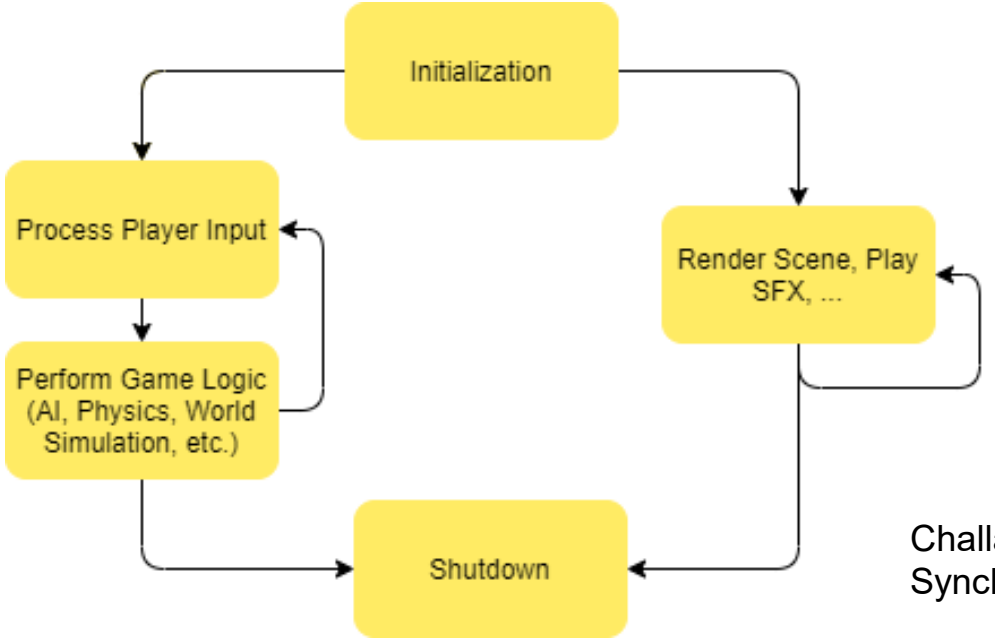
There can be many variations of the game loop, depending on a number of factors, most notably the target hardware. (Game Programming Algorithms and Techniques: A Platform-Agnostic Approach)

Simple Game Loop



Taken from Game Coding Complete Chapter 7

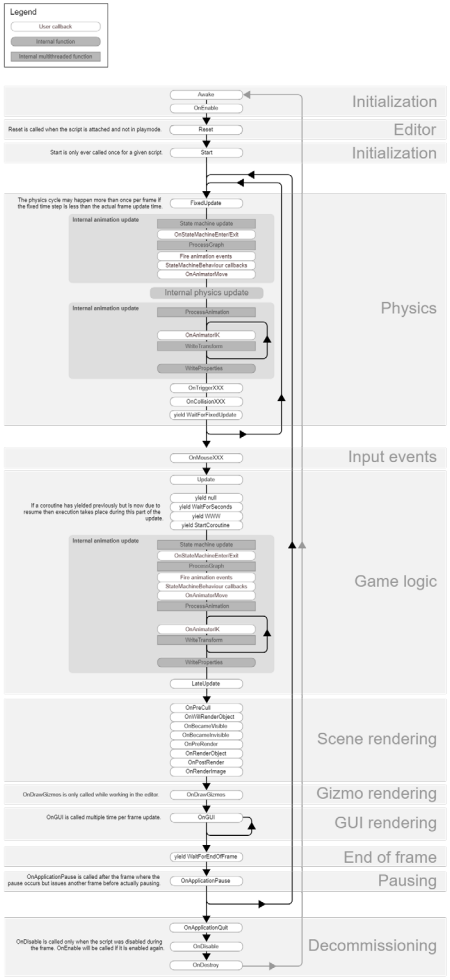
Multithreaded Game Loop



Challange:
Synchronization

Taken from Game Coding Complete Chapter 7

Unity Game-Loop



Taken from:
https://docs.unity3d.com/Manual/ExecutionOrder.html

More Game-Loop Patterns

- http://gameprogrammingpatterns.com/game-loop.html

Game Loop

Game Programming Patterns / Sequencing Patterns

Intent

Decouple the progression of game time from user input and processor speed.

Motivation

If there is one pattern this book couldn't live without, this is it. Game loops are the quintessential example of a "game programming pattern". Almost every game has one, no two are exactly alike, and relatively few programs outside of games use them.

To see how they're useful, let's take a quick trip down memory lane. In the olden days of computer programming when everyone had beards, programs worked like your dishwasher. You dumped a load of code in, pushed a button, waited, and got results out. Done. These were *batch mode* programs — once the work was done, the program stopped.

You still see these today, though thankfully we don't have to write them on punch cards anymore. Shell scripts, command line programs, and even the little Python script that turns a pile of Markdown into this book are all batch mode programs.

Interview with a CPU

Update Game

1. Calculate delta (time since last update call).
2. Call update for all game systems which needs to be updated per frame.
3. Call update of current game state.

- **Fixed Update**
  - Called in a fixed interval (delta always the same).
  - Importand for physics.
  - Render and update decoupled.
- **Variable Update**
  - Called every frame with variable delta.

What is a „Delta Time“?

Delta time or **delta timing** is a concept used amongst programmers in relation to hardware and network responsiveness. In graphics programming, the term is usually used for variably updating scenery based on the **elapsed time since the game last updated**, (i.e. the previous "frame") which will **vary depending on the speed of the computer**, **and how much work needs to be done in the program at any given time**.

Rendering

- Call render from all game systems which need to be drawn.
  1. 2D/3D scene
  2. Graphical User Interface
  3. Debug-Draw / Gizmos

FH Salzburg
MultiMediaTechnology

Basics Game Development

Introduction - End

Technology
Health
Media

© 2025 Fachhochschule Salzburg GmbH
www.fh-salzburg.ac.at